

UNIT 5 – Automation Testing & Selenium WebDriver

Exhaustive Study Notes covering Automation Frameworks, Selenium Architecture, Locators, XPath & Waits

1. Automation Testing Fundamentals

Automation Testing uses software scripts to execute test cases automatically, compare expected vs actual outcomes, and generate test reports.

Manual vs Automation Comparison: • Execution Speed: Manual = Slow | Automation = Fast. • Repetitive Regression: Manual = Tedious, error-prone | Automation = Ideal, repeatable. • Initial Setup Cost: Manual = Low | Automation = High. • Exploratory Testing: Manual = Ideal | Automation = Impossible.

Manual vs Automation Testing Pyramid & Decision Matrix

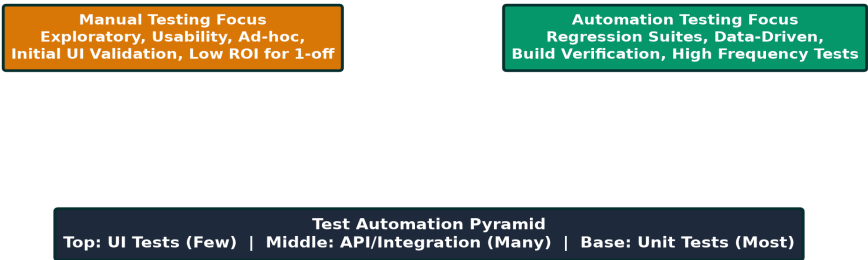


Diagram: Manual vs Automation Testing Pyramid & Decision Matrix.

2. Automation Concepts & Frameworks

Automation Framework Types: 1. Modular Framework: Breaks application into independent test scripts per module. 2. Data-Driven Framework: Separates test logic from test data (Excel / CSV / JSON). 3. Keyword-Driven Framework: Uses keywords (e.g. 'Click', 'InputText') stored in external tables. 4. Page Object Model (POM): Object-oriented design pattern creating Page classes for UI elements and Test classes for assertion execution.

Page Object Model (POM) Design Pattern Architecture

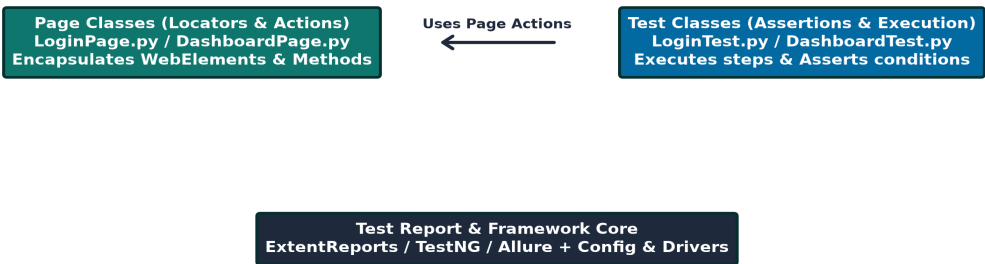


Diagram: Page Object Model (POM) Design Pattern Architecture.

3. Test Scripts, Assertions & Reports

- Assertions: Verify expected vs actual condition. Hard Assertion (stops test immediately on failure) vs Soft Assertion (logs failure and continues execution).
- Test Reports: Generate HTML reports (ExtentReports, Allure, TestNG HTML) containing test status (PASS/FAIL/SKIP), execution logs, and failure screenshots.

4. Selenium & Selenium WebDriver

Selenium is an open-source automation suite for web applications.

Selenium Components: • Selenium IDE: Chrome/Firefox browser extension for Record & Playback. • Selenium WebDriver: Programming API providing direct browser control via W3C protocol. • Selenium Grid: Enables parallel test execution across remote machines/browsers.

Selenium WebDriver Architecture (W3C Protocol)



Diagram: Selenium WebDriver Architecture (W3C Protocol).

5. Locators & XPath Strategies

Web Element Locators in Selenium: • `By.id("username")` (Fastest & most reliable) • `By.name("email")` • `By.className("btn-primary")` • `By.cssSelector("input[type='password']")` • `By.xpath("//input[@id='login']")`

Absolute vs Relative XPath: • Absolute XPath: Starts from root `/html/body/div[1]/form/input` (Fragile). • Relative XPath: Starts anywhere `/input[@name='email']` (Robust).

```

# Selenium Python Code Example
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://example.com/login")

# Locate elements & interact
driver.find_element(By.ID, "username").send_keys("testuser")
driver.find_element(By.XPATH, "//input[@type='password']").send_keys("Pass123!")
driver.find_element(By.CSS_SELECTOR, "button.btn-submit").click()
  
```

6. Handling Web Elements, Alerts, Frames & Windows

Handling Complex Web UI Elements:

```
# 1. Handling Dropdowns
from selenium.webdriver.support.ui import Select
select = Select(driver.find_element(By.ID, "country"))
select.select_by_visible_text("India")

# 2. Handling JavaScript Alerts
alert = driver.switch_to.alert
print(alert.text)
alert.accept() # Click OK

# 3. Handling Iframes
driver.switch_to.frame("frame_id")
# ... perform actions ...
driver.switch_to.default_content()

# 4. Handling Multiple Windows / Tabs
main_window = driver.current_window_handle
all_windows = driver.window_handles
for handle in all_windows:
    if handle != main_window:
        driver.switch_to.window(handle)
        break
```

7. Synchronization & Waits

Synchronization techniques handle asynchronous AJAX loading: • `Thread.sleep(5000)`: Hard pause (Not recommended, slows test suite). • Implicit Wait: Global timeout applied to all element lookups (`driver.implicitly_wait(10)`). • Explicit Wait: Targeted wait for specific condition before throwing exception (`WebDriverWait` + `ExpectedConditions`).

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Explicit Wait Example
wait = WebDriverWait(driver, 10)
element = wait.until(EC.element_to_be_clickable((By.ID, "submit-btn")))
element.click()
```